



UNIVERSIDAD DE BURGOS

Imágenes Táctiles para

Un Gripper Robótico

Alumno: Fernando Morán Fernández, con DNI 71378005N

Grado en Tecnologías Digitales para la Empresa

Profesor: Jaime Andres Rincon Arango

En Miranda de Ebro, a 27 de mayo de 2026

ÍNDICE

Introducción	3
Objetivos.....	3
Marco Teórico.....	3
Diseño y Metodología	3
Implementación	3
Resultados.....	3
FALLOS.....	5
Conclusiones	5
Manual de Ejecución	5
Repositorio GIT	5
Bibliografía.....	5

Introducción

Desarrollo un sistema distribuido que optimiza la percepción de un gripper robótico. El programa amplía matrices de presión táctil de baja resolución (16x16) para generar mapas de calor continuos, comunicando un proceso en C++ con otro en Python vía red.

Objetivos

- Leer y extraer datos desde archivos JSON mediante C++
- Desarrollar manualmente un algoritmo matemático de interpolación bilineal.
- Implementar una arquitectura de red Cliente-Servidor mediante HTTP POST.
- Generar automáticamente imágenes científicas usando Python.

Marco Teórico

- **Cliente-Servidor (HTTP):** Comunicación en la capa de aplicación usando formato JSON.
- **Modularidad:** Separación de procesos en memoria mediante archivos `.h` y `.cpp`.
- **Interpolación Bilineal:** Técnica matemática que calcula nuevos valores de presión promediando las distancias y pesos de los 4 pixeles vecinos más cercanos.

Diseño y Metodología

- **Cliente local (C++):** Realiza la carga computacional intensiva (procesamiento de arrays y matemáticas) y transmite los resultados.
- **Microservicio (Python):** Servidor ligero (`Flask`) diseñado para la recepción y renderizado rápido de gráficos.

Implementación

- **En C++:** Uso de la librería `nlohmann/json` para parsear los datos y `libcurl` para la transmisión por red. El escalado a 128 x 128 se logra con bucles anidados que protegen los límites de la matriz original.
- **En Python:** El decorador `@app.route` de Flask captura el JSON. `NumPy` reconstruye la matriz y Matplotlib genera el mapa táctil con la paleta de color térmica.

Resultados

El sistema validó y procesó el archivo de entrada sin errores. El cliente C++ empaquetó y envió las 50 iteraciones, mientras que el servidor

Python respondió con éxito, guardando localmente 50 archivos PNG de alta resolución.

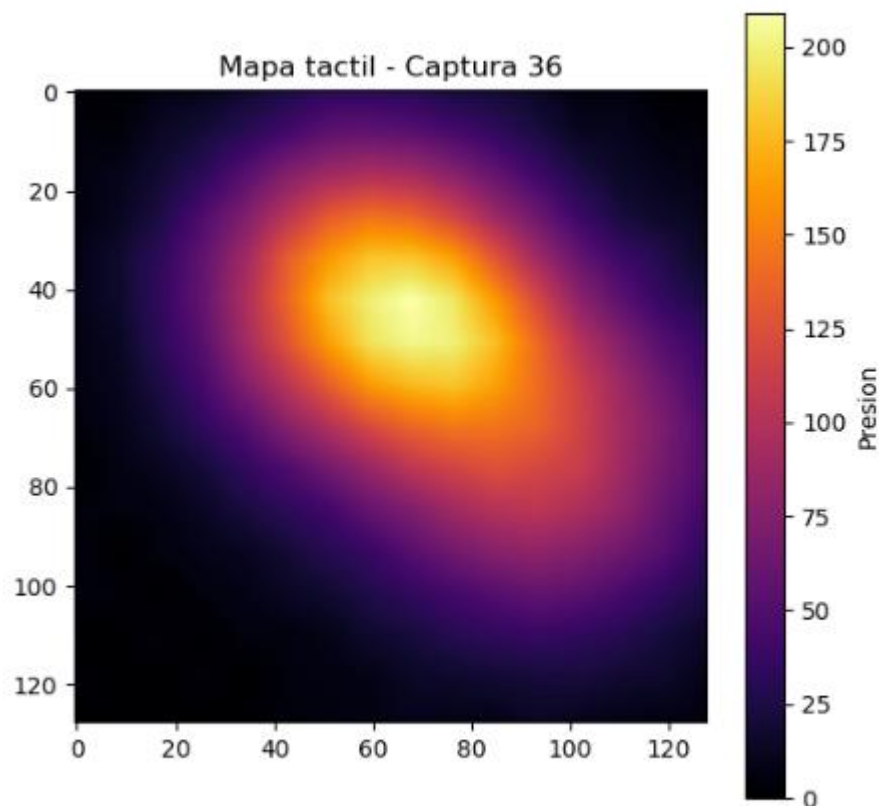
```
PROBLEMAS 8 SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
PS C:\Users\ferki\Desktop\UNI\2_CUATRI\Sistemas_operativos\practica_4> wsl
1
127.0.0.1 - - [27/May/2026 22:36:01] "POST /upload HTTP/1.1" 200 -
[OK] Imagen capture_37.png generada con éxito. (128x128)
127.0.0.1 - - [27/May/2026 22:36:01] "POST /upload HTTP/1.1" 200 -
[OK] Imagen capture_38.png generada con éxito. (128x128)
127.0.0.1 - - [27/May/2026 22:36:01] "POST /upload HTTP/1.1" 200 -
[OK] Imagen capture_39.png generada con éxito. (128x128)
127.0.0.1 - - [27/May/2026 22:36:02] "POST /upload HTTP/1.1" 200 -
[OK] Imagen capture_40.png generada con éxito. (128x128)
127.0.0.1 - - [27/May/2026 22:36:02] "POST /upload HTTP/1.1" 200 -
[OK] Imagen capture_41.png generada con éxito. (128x128)
127.0.0.1 - - [27/May/2026 22:36:02] "POST /upload HTTP/1.1" 200 -
[OK] Imagen capture_42.png generada con éxito. (128x128)
127.0.0.1 - - [27/May/2026 22:36:02] "POST /upload HTTP/1.1" 200 -
[OK] Imagen capture_43.png generada con éxito. (128x128)
127.0.0.1 - - [27/May/2026 22:36:02] "POST /upload HTTP/1.1" 200 -
[OK] Imagen capture_44.png generada con éxito. (128x128)
127.0.0.1 - - [27/May/2026 22:36:03] "POST /upload HTTP/1.1" 200 -
[OK] Imagen capture_45.png generada con éxito. (128x128)
127.0.0.1 - - [27/May/2026 22:36:03] "POST /upload HTTP/1.1" 200 -
[OK] Imagen capture_46.png generada con éxito. (128x128)
127.0.0.1 - - [27/May/2026 22:36:03] "POST /upload HTTP/1.1" 200 -
[OK] Imagen capture_47.png generada con éxito. (128x128)
127.0.0.1 - - [27/May/2026 22:36:03] "POST /upload HTTP/1.1" 200 -
[OK] Imagen capture_48.png generada con éxito. (128x128)
127.0.0.1 - - [27/May/2026 22:36:04] "POST /upload HTTP/1.1" 200 -
[OK] Imagen capture_49.png generada con éxito. (128x128)
127.0.0.1 - - [27/May/2026 22:36:04] "POST /upload HTTP/1.1" 200 -

PS C:\Users\ferki\Desktop\UNI\2_CUATRI\Sistemas_operativos\practica_4> wsl
Fallo al enviar a Python: Couldn't connect to server
[C++] Error al enviar la captura 49
fermorar@LAPTOP-SHJNG38H:/mnt/c/Users/ferki/Desktop/UNI/2_CUATRI/Sistemas_operativos/practica_4$ ./cliente
=== INICIANDO SISTEMA TACTIL ===

1. Leyendo archivo JSON...

2. Validando dimensiones de las matrices...
Exito: todas las 50 capturas tienen el tamaño correcto (16x16).

3. Interpolando y enviando prueba de la primera captura...
{"message": "Imagen 0 guardada", "status": "ok"}
[C++] Captura 0 enviada correctamente.
{"message": "Imagen 1 guardada", "status": "ok"}
[C++] Captura 1 enviada correctamente.
{"message": "Imagen 2 guardada", "status": "ok"}
[C++] Captura 2 enviada correctamente.
{"message": "Imagen 3 guardada", "status": "ok"}
[C++] Captura 3 enviada correctamente.
{"message": "Imagen 4 guardada", "status": "ok"}
[C++] Captura 4 enviada correctamente.
{"message": "Imagen 5 guardada", "status": "ok"}
[C++] Captura 5 enviada correctamente.
{"message": "Imagen 6 guardada", "status": "ok"}
[C++] Captura 6 enviada correctamente.
{"message": "Imagen 7 guardada", "status": "ok"}
[C++] Captura 7 enviada correctamente.
```



FALLOS

- El fallo que tuve fue no darme cuenta de que el servidor no estaba activado.

Conclusiones

Esta practica usamos C++ para el trabajo duro que son los cálculos matemáticos y Python para recibir datos y dibujar imágenes finales. Se transformo datos pobres y pixelados en mapas de precisión detallados y claros.

Manual de Ejecución

1. **Dependencias:** `sudo apt install nlohmann-json3-dev / libcurl4-openssl-dev / python3-flask / python3-numpy / python3-matplotlib`
2. **Servidor:** Ejecutar `python3 ServidorFlask.py` (dejar terminal abierta)
3. **Cliente:** Compilar con `g++ main.cpp procesamiento.cpp -o cliente -lcurl` y ejecutar `./cliente`

Repositorio GIT

- https://github.com/Fer851/Practica_4

Bibliografía

- Apuntes de sistemas operativos
- <https://curl.se/libcurl/>
- <https://matplotlib.org/>
- <https://flask.palletsprojects.com/en/stable/>
- <https://beej.us/guide/bgnet/>